

Sistemas Compostos de IA: Otimização de Respostas em Benchmarks Objetivos por meio de Orquestração Baseada no Ralph Loop

Hugo Emílio Nomura, Pedro Henrique Correia de Oliveira, Pedro Henrique Satoru Lima Takahashi, Vitor Monteiro Vianna

Ciência da Computação – Centro Universitário FEI, campus São Paulo

unifhnomura@fei.edu.br, unifpoliveira@fei.edu.br, unifptakahashi@fei.edu.br, unievvianna@fei.edu.br

Orientador: Charles Henrique Porto Ferreira

Departamento de Ciência da Computação, Centro Universitário FEI

cferreira@fei.edu.br

Resumo: Embora os *Large Language Models* (LLMs) continuem operando como geradores de texto, sua evolução recente não depende apenas do aumento de capacidade ou da complexidade matemática. Soluções modernas incorporam planejamento, refinamento e ciclos iterativos de raciocínio para melhorar a qualidade e a consistência das saídas. No entanto, o uso prolongado desses mecanismos amplia a janela de contexto e pode causar degradação de informação (*context rot*), reduzindo a confiabilidade das respostas.

Este trabalho propõe adaptar os princípios do Ralph Wiggum Loop para cenários gerais de *benchmarking*, com o objetivo de melhorar respostas por meio de chamadas iterativas ao próprio modelo, sem ferramentas externas. Para mitigar o *context rot*, a arquitetura preserva um comportamento *stateless* entre as fases, mantendo apenas os artefatos estritamente necessários a cada etapa.

O impacto empírico da abordagem será avaliado com os *benchmarks* MMLU e GPQA, comparando a acurácia de chamadas diretas ao modelo com a de execuções mediadas pelo *harness* proposto.

Palavras-chaves: *Large Language Models* (LLMs). Ralph Wiggum Loop. Sistemas Compostos de IA. Engenharia de Prompt. Decaimento de Contexto (*context rot*).

I. Introdução

Os *Large Language Models* (LLMs) consolidaram-se como o núcleo de uma nova geração de sistemas capazes de executar tarefas complexas de processamento de linguagem natural, incluindo geração de texto, resolução de problemas, planejamento intermediário e suporte à tomada de decisão. Apesar dessas capacidades, os modelos que servem como base para esses sistemas continuam sendo, em essência, geradores probabilísticos de *tokens*, operando a partir de uma entrada de texto e retornando como saída outro texto. Nesse contexto, parte significativa da evolução recente dos sistemas baseados em LLMs passou a ocorrer não apenas nos próprios modelos, mas também nas estruturas responsáveis por organizar sua utilização prática. Trabalhos recentes, como [1], afirmam que “*The performance of large language model (LLM) systems depends not only on model weights, but also on their harness: the code that determines what information to store, retrieve, and present to the model*”, evidenciando o papel dos chamados *harnesses* na ampliação das capacidades desses sistemas. Ferramentas modernas, como o *ChatGPT*, incorporam tais mecanismos para adicionar camadas de coordenação, decomposição de tarefas, validação intermediária e orquestração de múltiplas inferências, permitindo que o modelo execute fluxos significativamente mais sofisticados do que uma única chamada linear de entrada e saída.

Abordagens baseadas em uma única chamada ao modelo, nas quais toda a resposta é produzida em uma única

etapa de inferência, apresentam limitações importantes em tarefas que exigem raciocínio complexo, múltiplas etapas de decisão ou validações intermediárias. Como resposta a esse problema, diferentes linhas de pesquisa passaram a explorar fluxos estruturados de *thinking*, nos quais o processo de inferência é explicitamente decomposto em etapas sucessivas de raciocínio, planejamento, execução e refinamento. Trabalhos baseados em *Chain-of-Thought* [2], *Tree of Thoughts* [3] e na técnica de *Least-to-Most* [4] demonstram que a decomposição explícita do raciocínio favorece significativamente a resolução de tarefas complexas. De maneira semelhante, abordagens como *ReAct* [5] combinam raciocínio e ações iterativas, enquanto métodos como *Reflexion* [6], *Self-Debugging* [7] e *Self-Refine* [8] exploram ciclos sucessivos de avaliação, crítica e refinamento da própria resposta gerada. Em conjunto, esses trabalhos indicam que ganhos de desempenho não dependem exclusivamente do tamanho do modelo ou da quantidade de *tokens* gerados, mas também da maneira como o processo de inferência é estruturado, coordenado e validado ao longo da execução.

Apesar dos ganhos observados nessas arquiteturas iterativas compostas por agentes, ciclos de reflexão ou *pipelines* sucessivos, tais abordagens também introduzem novos desafios relacionados ao gerenciamento do contexto acumulado durante a inferência. Em muitos desses sistemas, o histórico completo de interações permanece armazenado na janela de contexto ao longo de toda a execução como mecanismo de preservação da continuidade do raciocínio.

Embora essa estratégia favoreça rastreabilidade e coerência entre etapas intermediárias, ela também aumenta progressivamente a dificuldade de recuperação de informações relevantes conforme o contexto cresce. Liu et al. [9], por exemplo, demonstram que ocorre perda de desempenho em tarefas que exigem localizar informações relevantes em sequências extensas. Trabalhos recentes também passaram a discutir fenômenos associados à degradação progressiva do contexto em execuções longas, incluindo compressão implícita de informações, perda de relevância contextual e aumento de inconsistências ao longo da inferência. Esses efeitos vêm sendo frequentemente associados, ao conceito de *context rot*, termo utilizado por Hong et al. [10] para descrever a degradação gradual da qualidade das respostas conforme o volume de contexto acumulado aumenta.

Além do avanço acadêmico dessas abordagens, a indústria de engenharia de software também passou a investir intensamente em arquiteturas de *deep thinking* e em *harnesses* especializados para ampliar a capacidade operacional de LLMs em tarefas complexas de desenvolvimento. Ferramentas recentes, como *Claude Code*, *GPT Codex*, *OpenCLI* e *Devin*, exemplificam essa tendência ao combinar planejamento, decomposição de tarefas, validação iterativa e execução coordenada de múltiplas inferências. Nesse contexto, uma das abordagens que ganhou notoriedade foi o *Ralph Wiggum Loop* [11], popularizado por Geoffrey Huntley no contexto de engenharia de software. Em sua formulação original, entretanto, o modelo não foi concebido para cenários generalistas, mas especificamente para fluxos de desenvolvimento baseados em conceitos próprios da engenharia de software, como definição de funcionalidades, requisitos, critérios de aceite e integração com sistemas de versionamento, como o *Git*. Ainda assim, sua estrutura apresenta dois princípios centrais relevantes para os problemas discutidos na literatura de LLMs: a decomposição do processo em tarefas independentes, reduzindo a necessidade de reaproveitamento contínuo do contexto entre etapas, e a execução iterativa de cada tarefa até que critérios mínimos de qualidade sejam satisfatoriamente atingidos.

Este trabalho parte de três premissas consolidadas na literatura: que abordagens de *thinking* podem melhorar a qualidade das respostas de LLMs em cenários específicos, como os de tarefas complexas; que tais abordagens introduzem problemas associados ao crescimento excessivo do contexto acumulado, fenômeno descrito como *context rot*; e que o *Ralph Wiggum Loop* [11], concebido por Huntley para fluxos de engenharia de software, oferece princípios estruturais capazes de mitigar o problema de *context rot* por meio de decomposição de tarefas e isolamento contextual entre etapas. A partir dessas premissas, propõe-se a adaptação do *Ralph Wiggum Loop* para um contexto generalista de resolução de tarefas em linguagem natural a partir de um *prompt* de entrada fornecido pelo usuário. Diferentemente da proposta original, centrada em desenvolvimento de software, definições de funcionalidades, critérios de aceite baseados em requisitos funcionais e integração com ferramentas de versionamento (como *Git*), a adaptação proposta investiga se, ao adaptar a estrutura do *Ralph Wiggum*

Loop a um fluxo completo de *thinking* aplicável à resolução de perguntas gerais e à exploração de problemas diversos e de diferentes complexidades, também haverá melhora na qualidade dessas respostas quando comparadas a uma única etapa de inferência.

Para avaliar essa hipótese de maneira objetiva, reproduzível e quantitativa, o trabalho compara diretamente duas estratégias de inferência: (i) uma chamada tradicional a um modelo de LLM, na qual toda a resposta é produzida em uma única etapa de inferência; e (ii) uma execução mediada pelo *harness* proposto, que adapta o *Ralph Wiggum Loop* para cenários generalistas. O objetivo experimental é medir em que magnitude a aplicação desse fluxo estruturado de *thinking* melhora a qualidade das respostas em relação à inferência singular tradicional. Para permitir essa quantificação, os experimentos utilizam *benchmarks* compostos por questões e respostas previamente definidas, estratégia amplamente adotada para mensuração de ganhos em arquiteturas de raciocínio e orquestração de LLMs, e usada em trabalhos como *ReAct* [5] e *Beyond the Strongest LLM* [12]. Os experimentos são conduzidos com diferentes modelos da família Llama 3.1, com diferentes quantidades de parâmetros utilizados na execução do modelo, representando distintos níveis de complexidade computacional, e avaliados por meio dos *benchmarks* MMLU [13] e GPQA [14], sendo a acurácia, medida pela quantidade de acertos entre a alternativa selecionada pelo sistema de LLM e o gabarito oficial.

II. Referencial Bibliográfico

Esta seção apresenta os conceitos fundamentais necessários para a compreensão do trabalho proposto. São definidos os termos técnicos centrais utilizados ao longo do texto, descritas as principais estratégias de raciocínio e refinamento aplicadas a modelos de linguagem, e introduzido o paradigma de orquestração adotado neste trabalho.

A. Modelos de Linguagem e a Arquitetura de Harness

Embora os *Large Language Models* (LLMs) [15] sejam amplamente consolidados como geradores probabilísticos de *tokens* baseados na arquitetura *Transformer* [16], a evolução recente na aplicação dessas tecnologias migrou de chamadas diretas e lineares para o uso de estruturas de orquestração mais robustas. Um detalhamento teórico e histórico aprofundado sobre o funcionamento desses modelos de linguagem pode ser encontrado no levantamento conduzido por Zhao et al. [17].

Na prática, a organização dessas interações é viabilizada por meio de um *harness*, termo que se refere à estrutura lógica e às regras de orquestração que envolvem um modelo de linguagem para guiar seu processo de inferência, funcionando como uma camada de coordenação entre o *prompt* inicial do usuário e a saída do modelo [12], [18]. Um *harness* pode incluir etapas de planejamento, decomposição de tarefas, validação intermediária e composição de respostas finais. Variações na arquitetura do *harness* podem causar impactos significativos no desempenho do sistema como um todo, mesmo quando o modelo base permanece o

mesmo.

Esse conceito está diretamente relacionado à noção de *Sistemas Compostos de IA*, definida por Zaharia et al. [18] como arquiteturas que resolvem tarefas de inteligência artificial por meio de múltiplos componentes que interagem entre si, incluindo chamadas a modelos, recuperadores de informação e ferramentas externas. Segundo essa perspectiva, os ganhos mais significativos em aplicações de IA contemporâneas passaram a depender da engenharia de sistemas que combinam componentes especializados em fluxos coordenados, em vez de se apoiarem unicamente no aumento de escala dos modelos base [18].

Essa tendência se manifesta em ferramentas industriais que utilizam LLMs integrados a *harnesses* especializados para tarefas de engenharia e automação. Exemplos incluem o Claude Code [19], interface de linha de comando voltada ao desenvolvimento assistido por IA; o GPT Codex [20], otimizado para geração e compreensão de código; e o OpenCLI [21], voltado à automação de comandos de terminal. Essas ferramentas demonstram como a combinação de planejamento, decomposição de tarefas e validação iterativa expande a capacidade computacional útil dos modelos base.

B. Estratégias de Raciocínio e Refinamento

A literatura demonstra que a qualidade da inferência de um LLM pode ser melhorada de forma significativa quando o processo é estruturado em fases [2], [3]. O método *Chain-of-Thought* [2] e sua generalização *Tree of Thoughts* [3] mostraram a importância de induzir o modelo a explicitar passos intermediários de raciocínio. Abordagens como *ReAct* [5] ampliaram esse princípio ao combinar o raciocínio interno do modelo com a capacidade de invocar ferramentas externas, permitindo corrigir alucinações por meio de observações diretas do ambiente.

De forma complementar, pipelines baseados em ciclos de *feedback* iterativo também apresentam ganhos consistentes. O framework *Reflexion* [6] introduziu ciclos nos quais o modelo atua como seu próprio crítico, revisando saídas com base em reforço verbal externo. A abordagem de *Self-Debugging* [7] demonstrou que LLMs podem identificar e corrigir seus próprios erros quando guiados de forma sistemática. Na mesma direção, o método *Self-Refine* [8] consolidou a eficácia do refinamento iterativo ao demonstrar que modelos podem gerar *feedback* crítico sobre suas próprias respostas e utilizá-lo para melhorar as saídas de forma sucessiva, contornando limitações da geração em uma única etapa. Essa capacidade de autocrítica e revisão contínua constitui uma premissa fundamental para a proposta deste trabalho.

Além dessas abordagens, métodos focados na decomposição de tarefas complexas também contribuem para a redução de erros lógicos e omissões. Técnicas de *prompting* baseadas em decomposição, como *Least-to-Most* [4], *Decomposed Prompting* [22] e *Plan-and-Solve* [23], sustentam que separar o planejamento da execução e dividir problemas em subproblemas menores melhora o desempenho do modelo, especialmente quando combinado com mecanismos de validação intermediária.

C. Degradação de Contexto

Embora as técnicas apresentadas nas subseções anteriores tenham demonstrado eficácia individual, muitas delas dependem do acúmulo contínuo de estado em uma única janela de contexto, o que pode agravar problemas de degradação à medida que o histórico cresce. Liu et al. [9] demonstraram empiricamente que LLMs apresentam perda significativa de desempenho quando precisam recuperar informações relevantes posicionadas no meio de sequências longas, fenômeno descrito como “perdidos no meio” (*lost in the middle*).

Na prática, esse comportamento se manifesta de diversas formas: compressão implícita de informações, perda de relevância contextual, aumento de inconsistências e maior incidência de alucinações em execuções prolongadas [9], [17]. A literatura contemporânea frequentemente associa esse conjunto de efeitos ao conceito de *context rot*, referindo-se à degradação progressiva da qualidade das respostas conforme o contexto acumula ruído, redundâncias e informações que competem pela atenção do modelo [9].

D. O Ralph Wiggum Loop

Diante dos problemas de degradação de contexto, o *Ralph Wiggum Loop* [11] surgiu na comunidade de engenharia de software como um paradigma de orquestração que aborda diretamente essa limitação. Idealizado por Geoffrey Huntley, o paradigma parte de um conjunto de requisitos fornecidos como entrada e os decompõe em histórias de usuário, conceito consolidado na disciplina de engenharia de software. Cada história é tratada de forma atômica: o modelo implementa uma solução, executa testes e verifica se os critérios de aceite foram satisfeitos; caso contrário, o ciclo se repete até que a história seja concluída com êxito. Ao término de cada história, seus resultados não são carregados diretamente para a execução seguinte, evitando o acúmulo progressivo de estado que caracteriza arquiteturas tradicionais de agentes.

Um aspecto central dessa abordagem é o princípio de *Fresh Context* [11]: ao iniciar cada nova história, o contexto de execução é completamente reinicializado. Antes de qualquer instrução sobre a nova história, os aprendizados gerais extraídos das histórias anteriores são injetados como primeira entrada nesse novo contexto, de forma estruturada e explícita. Dessa maneira, o paradigma preserva o conhecimento relevante acumulado sem herdar o ruído, as redundâncias e as informações obsoletas que degradam progressivamente a qualidade das respostas em execuções longas, mitigando ativamente o fenômeno de *context rot* [10].

Em sua formulação original, o *Ralph Wiggum Loop* foi concebido especificamente para fluxos de engenharia de software, com etapas voltadas à definição de funcionalidades, requisitos, critérios de aceite e integração com sistemas de versionamento como *Git*. Ainda assim, seus princípios de execução *stateless* e validação iterativa apresentam relevância direta para os problemas discutidos na literatura de LLMs. A validação acadêmica dessa abordagem foi reforçada pelo estudo de McComb et al. [24], que avaliou

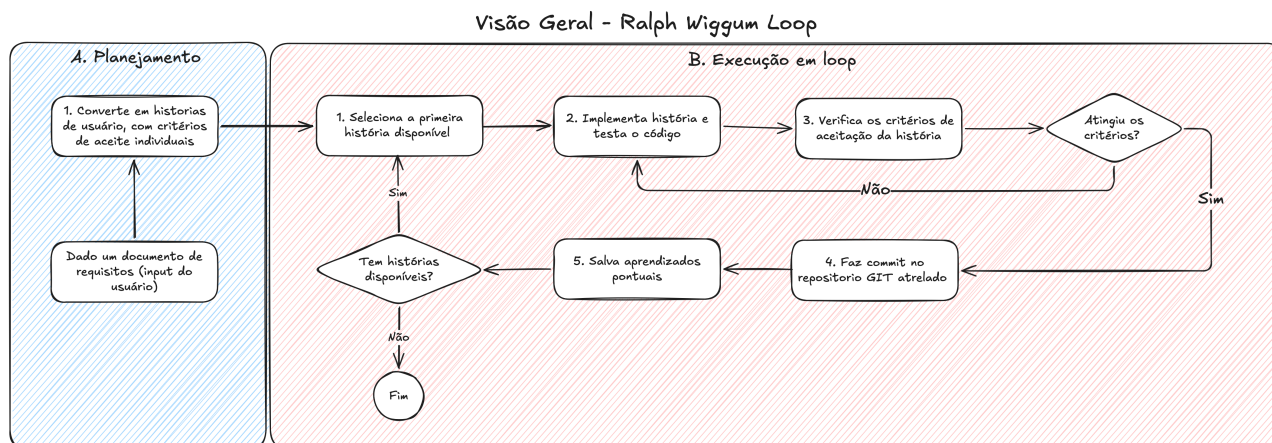


Figura 1. Ralph Wiggum Loop original.

o impacto de agentes baseados no *Ralph Wiggum Loop* na resolução de problemas complexos de design de engenharia, confirmando a eficácia do paradigma *stateless* em contextos que exigem raciocínio estruturado e iterativo. Este trabalho se apoia nesses fundamentos para justificar o uso do paradigma, adaptando sua orquestração para a resolução de questões de propósito geral em múltiplos domínios do conhecimento.

III. Metodologia

A adaptação do *Ralph Wiggum Loop* para um contexto generalista de resolução de perguntas em linguagem natural exige ajustes estruturais em relação à formulação original, que foi concebida exclusivamente para fluxos de engenharia de software. A Figura 2 apresenta uma visão geral do fluxo proposto, que preserva os princípios centrais do paradigma original, representado na Figura 1.

Em linhas gerais, propomos uma conversão do conceito de histórias de usuário para tarefas, o que nos permite manter um fluxo muito próximo do original, contendo as etapas de “Planejamento” e “Execução em Loop”. Entretanto, passa a ser necessário considerar a criação de um retorno ao usuário, algo que não constitui uma preocupação no *Ralph Wiggum Loop* original. Dessa forma, incorporamos uma nova fase após fluxo de execução, denominada “Criação da Resposta Final”, que contém toda a lógica necessária para agrupar os resultados produzidos pelas tarefas anteriores em uma resposta que apresente o conteúdo e o formato esperados pelo usuário que realizou a solicitação.

A. Planejamento

O planejamento é o início de todo o fluxo. Aqui assumimos que estamos recebendo uma pergunta e vamos iniciar o fluxo de *Thinking* da aplicação. Assim como exemplificado na Figura 2, o planejamento possui somente duas etapas:

- Exemplo pratico de input passado pelo usuario:

Qual dos seguintes números é simultaneamente primo, par e maior que 2?

A) 2

B) 4

C) 6

D) Nenhuma das anteriores

A.1 Defição dos critérios de aceite gerais

- Explicação da responsabilidade: - Essa é uma fase que não era necessaria no contexto de desenvolvimento de software, ja que a cada tarefa que era entregue, um commit era feito salvando o progresso, e quando todas as terafas tivessem encerrado, o desenvolvimento tambemn estaria pronto. - Agora o cenario muda, apos todas as tarefas serem todas concluidas, vai ser necessario enviar uma resposta final usando o resultado das tarefas geradas, e esses criterios devem trazer definições que vão ser usadas. - Vão ser definidos pontos como: Idioma a ser usado na resposta final, Qual o formato que deve ser construida a resposta final, topicos chaves do conteudo que devem estar presentes na resposta final

- Como vai ser feito: - Criação de um template FIXO, onde vai ser injetado o input passado pelo usuario. Esse template deve conter um prompt solicitando a definição de uma lista de criterios de aceite que vão ser resgatados na fase de criação da resposta final. - Deve ficar claro que o tempelate dessa fase vai somente pedir aquebra e fornecer exemplos dessas quebras - O template vai pedir para que ocorra definição de: - Idioma a ser usado na resposta final - Qual o formato que deve ser construida a resposta final - Topicos chaves do conteudo que devem estar presentes na resposta final

- Equação de funcinamento da fase: - Pergunta do usuario + Template A.1 = Resultado Fase A.1 - O Resultado Fase A.1 deve ser armazenado localmente em um arquivo ou na memoria do programa

- Exmplo pratico: - Assumindo input passado no inicio o LLM deveria retornar algo como: - "A pergunta deve ser respondida em Portugues do Brasil", "Reponda somente a alternativa, sem conter explicações adicionais", "Garanta que o valor encontrado ao final bate com uma das alternativas"

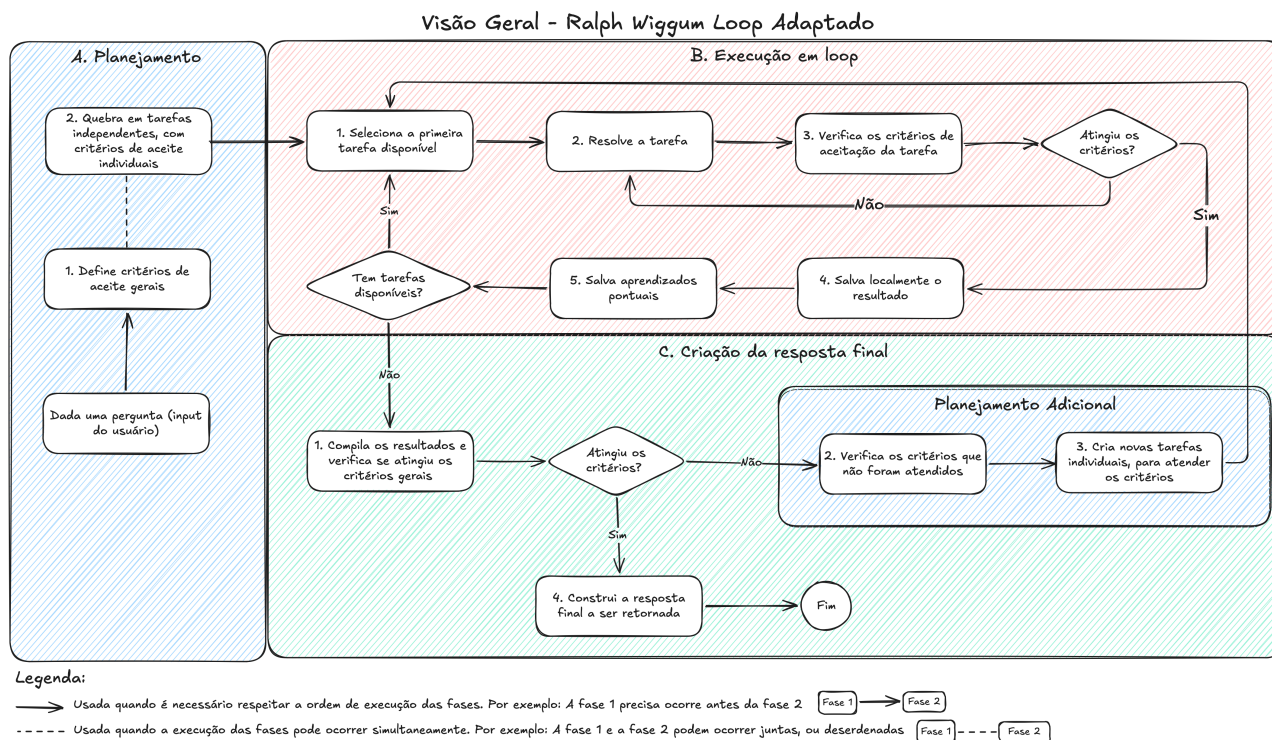


Figura 2. Visão geral do *Ralph Wiggum Loop* adaptado para resolução de perguntas em linguagem natural.

A.2 Quebra em tarefas de objetivo único com critérios de aceite individuais

- Explicação da responsabilidade: - Quebrar a pergunta do usuário em pequenas fases. - Cada fase precisa ter um critério de aceite, ou seja, o conteúdo que esperamos que seja gerado para que a fase possa ser considerada como "Concluída- Aqui a ideia é criar fases que gerem insumos capazes de gerar uma resposta final ao se juntar.

- Como vai ser feito: - Criação de um template FIXO, onde vai ser injetado o input passado pelo usuário. Esse template deve conter um prompt solicitando a quebra da pergunta fornecida como input pelo usuário em tarefas. - Ainda nesse template, deve ser especificado que as tarefas de objetivo único, nesse contexto, significam ter uma única responsabilidade pontual, como "pegar uma fórmula" ou "buscar o dado X", mas nunca "Buscar pela fórmula X, substituir pelos valores A, B, C e executar a conta matemática para encontrar o resultado". - Além de ter um único objetivo, a tarefa deve ser o mais detalhada possível, para que não ocorram desvios de objetivo. - Nesse prompt de quebra também deve ficar claro que o contexto do que foi feito não vai pra próxima fase, somente serão salvos pontos específicos que podem ser úteis para fases posteriores, como "A fórmula X é usada para calcular o consumo de energia elétrica- Preciso que fique claro que o objetivo dessas fases é gerar insumos para que a tarefa possa ser resolvida - Também nesse template deve ser definido que toda tarefa deve ter pelo menos um critério de aceite, que valida se a resposta final gerada contém o conteúdo necessário para encerrar a fase

- Equação de funcionamento da fase: - Pergunta do

usuário + Template A.2 = Resultado Fase A.2 - O Resultado Fase A.2 deve ser armazenado localmente em um arquivo ou na memória do programa

- Exemplo prático: - Assumindo input passado no início o LLM deveria retornar algo como: - Tarefa 1: - Tarefa: - "Dadas as opções (2, 4, 6), identifique quais desses números são primos- Critérios de aceite: - "Deve retornar a lista de números primos encontrados nas alternativas.- "Deve incluir uma breve explicação da regra de identificação de números primos.- "A resposta não deve tentar resolver a questão final.- "A resposta não deve avaliar nenhuma outra condição"

- Tarefa 2: - Tarefa: - "Dadas as opções (2, 4, 6), identifique quais desses números são pares- Critérios de aceite: - "Deve retornar a lista de números pares encontrados nas alternativas.- "Deve incluir uma breve explicação da regra de identificação de números pares.- "A resposta não deve tentar resolver a questão final.- "A resposta não deve avaliar nenhuma outra condição"

- Tarefa 3: - Tarefa: - "Dadas as opções (2, 4, 6), identifique quais desses números são estritamente maiores que 2.- Critérios de aceite: - "Deve retornar a lista de números maiores que 2 encontrados nas alternativas.- "Deve incluir uma breve justificativa matemática para a comparação estabelecida.- "A resposta não deve tentar resolver a questão final.- "A resposta não deve avaliar nenhuma outra condição"

- Nota. O formato usado é somente de exemplo para mostrar o conteúdo que deve ser gerado ao fim

B. Execução em Loop

A fase de execução preserva a estrutura iterativa do paradigma original, mas adapta o mecanismo de conclusão

de cada unidade de trabalho. Na formulação original, a conclusão de uma história de usuário é verificada por meio da execução de testes automatizados e da análise dos critérios de aceite associados ao código produzido. No fluxo adaptado, como não há código a ser testado, o modelo simplesmente resolve a tarefa selecionada e, em seguida, verifica se o resultado satisfaz os critérios de aceite individuais definidos no planejamento. Caso os critérios não sejam atingidos, o modelo repete a resolução da mesma tarefa até que sejam satisfeitos.

Ao concluir uma tarefa com sucesso, o modelo salva o resultado localmente e registra os aprendizados pontuais obtidos ao longo daquela execução. Esses aprendizados correspondem ao mecanismo de *Fresh Context* do paradigma original: antes de iniciar a execução da tarefa seguinte, o contexto é reinicializado e os aprendizados acumulados são injetados como primeira entrada do novo contexto, de forma estruturada e explícita. Esse mecanismo garante que o conhecimento relevante seja preservado sem que o ruído e as redundâncias das etapas anteriores sejam herdadas, mitigando ativamente o fenômeno de *context rot* [10]. Para o armazenamento local dos resultados, adota-se o formato JSON por conveniência de manipulação programática, embora qualquer formato que permita recuperação posterior seja funcionalmente equivalente. O ciclo se repete até que todas as tarefas planejadas sejam concluídas.

C. Criação da Resposta Final

A terceira fase constitui uma adição exclusiva do fluxo adaptado, sem correspondência direta na formulação original. No *Ralph Wiggum Loop* concebido para engenharia de software, o produto final é o próprio código implementado e versionado ao longo das histórias; não há, portanto, necessidade de uma etapa de síntese posterior. No fluxo adaptado, os resultados das tarefas individuais são fragmentos de raciocínio que precisam ser compilados em uma resposta coesa e completa.

Após o encerramento do loop, o modelo compila os resultados armazenados e verifica se as informações produzidas são suficientes para satisfazer todos os critérios de aceite gerais definidos no planejamento. Se os critérios forem integralmente atendidos, o modelo constrói a resposta final e a retorna ao usuário. Caso contrário, o sistema infere que o planejamento inicial foi insuficiente para cobrir todos os aspectos da pergunta e inicia uma fase de *Planejamento Adicional*: o modelo identifica os critérios ainda não atendidos, cria novas tarefas individuais para supri-los e retorna à fase de Execução em Loop. Esse mecanismo garante que lacunas de planejamento sejam identificadas e corrigidas antes que a resposta final seja construída, preservando a completude e a coerência da saída.

IV. Cronograma do projeto

V. Referências

Referências

- [1] Y. Lee, R. Nair, Q. Zhang, K. Lee, O. Khattab e C. Finn, “Meta-Harness: End-to-End Optimization of

Model Harnesses,” *arXiv preprint arXiv:2603.28052*, 2026.

- [2] J. Wei et al., “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models,” *arXiv preprint arXiv:2201.11903*, 2022.
- [3] S. Yao et al., “Tree of Thoughts: Deliberate Problem Solving with Large Language Models,” *arXiv preprint arXiv:2305.10601*, 2023.
- [4] D. Zhou et al., “Least-to-most prompting enables complex reasoning in large language models,” *arXiv preprint arXiv:2205.10625*, 2022.
- [5] S. Yao et al., “ReAct: Synergizing Reasoning and Acting in Language Models,” *arXiv preprint arXiv:2210.03629*, 2022.
- [6] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan e S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *arXiv preprint arXiv:2303.11366*, 2023.
- [7] X. Chen, M. Lin, N. Schärli e D. Zhou, “Teaching Large Language Models to Self-Debug,” *arXiv preprint arXiv:2304.05128*, 2023.
- [8] A. Madaan et al., “Self-Refine: Iterative Refinement with Self-Feedback,” *arXiv preprint arXiv:2303.17651*, 2023.
- [9] N. F. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *Transactions of the Association for Computational Linguistics*, v. 12, pp. 277–294, 2024.
- [10] K. Hong, A. Troynikov e J. Huber, “Context Rot: How Increasing Input Tokens Impacts LLM Performance,” *Chroma*, rel. técn., jul. de 2025. endereço: <https://trychroma.com/research/context-rot>
- [11] G. Ghuntley, *Ralph Loop: A Structured Orchestration Pattern for LLM Inference*, Online. Disponível em: <https://ghuntley.com/specs/ralph/>, Acesso em: abr. 2026, 2025.
- [12] Z. Wang et al., “Beyond the Strongest LLM: Multi-Turn Multi-Agent Orchestration vs. Single LLMs on Benchmarks,” *arXiv preprint arXiv:2509.23537*, 2025.
- [13] D. Hendrycks et al., “Measuring Massive Multitask Language Understanding,” em *International Conference on Learning Representations*, 2021.
- [14] D. Rein et al., “GPQA: A Graduate-Level Google-Proof Q&A Benchmark,” *arXiv preprint arXiv:2311.12022*, 2023.
- [15] T. B. Brown et al., “Language Models are Few-Shot Learners,” em *Advances in Neural Information Processing Systems*, 2020.
- [16] A. Vaswani et al., “Attention is All You Need,” em *Advances in Neural Information Processing Systems*, vol. 30, 2017.

Tabela I. Cronograma das atividades do projeto.

Atividade	fev	mar	abr	mai	jun	jul	ago	set	out	nov
Revisão bibliográfica e refinamento da proposta	X	X	X	X						
Definição da arquitetura		X	X	X	X					
Desenvolvimento do Paper		X	X	X	X	X	X	X	X	X
Implementação prática do código				X	X	X	X	X	X	
Análise final									X	X

- [17] W. X. Zhao et al., “A Survey of Large Language Models,” *arXiv preprint arXiv:2303.18223*, 2023.
- [18] M. Zaharia et al., *The Shift from Models to Compound AI Systems*, Berkeley Artificial Intelligence Research Blog. Disponível em: <https://bair.berkeley.edu/blog/2024/02/18/compound-ai-systems/>, Acesso em: mai. 2026, 2024.
- [19] Anthropic, *Claude Code*, <https://www.anthropic.com/claude-code>, 2026.
- [20] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv preprint arXiv:2107.03374*, 2021.
- [21] *OpenCLI*, <https://opencli.ai/>, 2026.
- [22] T. Khot et al., “Decomposed prompting: A modular approach for solving complex tasks,” *arXiv preprint arXiv:2210.02406*, 2022.
- [23] L. Wang et al., “Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models,” *arXiv preprint arXiv:2305.04091*, 2023.
- [24] C. McComb et al., “Supervising Ralph Wiggum: Exploring a Metacognitive Co-Regulation Agentic AI Loop for Engineering Design,” *arXiv preprint arXiv:2603.24768*, 2026.